Beef

a messaging app

Initial Report

Mentors

AJ

Cookout Software

Cole Godfrey
Ethan Goode
Ben Tanabe

CptS 322 Software Engineering Principles I
Spring 2026

Table of Contents

I. Project Outline **2**
I.1. Abstract
I.2. Parameters of Assignment
I.3. Learning Outcomes

II. Description and Clarification **2**
II.1. Introduction
II.2. Background and Related Work
II.3. Project Overview
II.3.1. Goals
II.4. Client and Stakeholder Identification
II.5. Team Members and Bios
II.5.1. Member: Cole Godfrey
II.5.2. Member: Ethan Goode
II.5.3. Member: Ben Tanabe

III. Requirements and Specifications **7**
III.1. Introduction
III.2. System Requirements Specification
III.2.1. Use Cases
III.2.2. Functional Requirements
III.2.3. Non-functional Requirements
III.2.4. Standards
III.3. System Evolution

IV. User Interface Design
IV.1. System Overview
IV.2. Architecture Design
IV.2.1 Overview
IV.2.2 Subsystem Decomposition
IV.3. Data Design
IV.3.1 Database Choice
IV.3.2 Database Design
IV.4. User Interface Design
IV.5. Constraints

I. Project Outline

I.1. Abstract

Inspired by one of the largest messaging apps in the world, Discord, our team set off to create a similar application called Beef, with the main differentiator being the integration of a zero-trust architecture.

Although for CPTS 322 we do not have any direct clients, we are working on Beef mainly to improve our knowledge of the MERN stack, implement various elements relating to zero-trust architecture and other security features, and gain experience working as a team to develop a full-stack social messaging application.

I.2. Parameters of Assignment

- Text messaging
- Audio calls
- User Accounts
- Beef-themed interface
- Guild/Group Chats
- Security
    - Zero Trust Architecture
    - OAuth user sign-in functionality

I.3. Learning Outcomes

- Increase our understanding of web app development.
- Learn frontend design.
- Understand and implement ZTA.
- Deepen understanding of the MERN framework.

II. Description and Clarification

II.1. Introduction

Beef is a voice- and text-based messaging app heavily inspired by Discord. Running as microservices on dedicated servers, Beef is intended to give our developers a wide range of experience in a modern tech stack while also functioning as a powerful backend. Utilizing popular technologies like MERN and Docker, our goal is to develop a robust and horizontally scalable messaging service wrapped in a tasteful meat-themed UI.

II.2. Background and Related Work

This project is inspired by Discord, which is a voice, video, and text communication platform to help people interact and connect with one another. It has a community focus, trying to accommodate everything from gaming clubs to school clubs.

Our project aims to focus less on accommodating a wide range of users and focus on security. By implementing Zero Trust Architecture, we plan to significantly increase security for our users and target specifically those who value that security.

Discord has had a data breach as recently as October 2025, which leaked user information such as Usernames, email addresses, messages, and government ID images. Beef utilizes ZTA to ensure a lower frequency of data breaches.

II.3. Project Overview

The project is a web application that can be used by anyone who values security to send messages, create guilds/groups, and call others both privately and in a large group, all while being protected by a Zero Trust Architecture design. To be honest zero trust architectures are complex in nature and our team has no practical experience. So although the system may not fully qualify, we will at least have some zero-trust-adjacent security.

One of the core principles of zero trust is that traffic is never trusted based on its location within a network (even inside the firewall/proxy). We will implement ZTA by having a PEP (Policy

Enforcement Point) between each module and a PDP (Policy Decision Point) to govern how and when modules may communicate. When traffic is sent between services, a few steps are followed: Both parties authenticate their identities, the PEP verifies with the PDP whether the traffic is permitted, and an mTLS-encrypted session will be established if it is. If the traffic is not permitted, the PEP will block it. The PDP makes a final judgement on whether traffic should be permitted based on the attempted actions and previous behavior (among other statistics) of the involved parties.

This system works well for transactional/latency-insensitive traffic (such as user creation and joining new guilds). However: we haven't tested it yet, but we suspect that other traffic like high volume message streams or low latency audio channels may prove difficult for this architecture to support. With high volume message traffic, the PDP may become a choke point if sessions are created on a message-by-message basis (with every session requiring approval). Since latency isn't a big concern in the direct messaging world, one solution may be to gather messages and send them in bulk per session (requiring only a single PDP query). Audio calls on the other hand require high-stability low-latency traffic, and getting approval from the PDP may add too much temporal distance. As such, we may need to configure the PEPs to authorize long-running sessions between endpoints for the duration of a given voice chat.

We aren't yet sure what PEPs we will use or if they will be capable of long sessions, so experimentation will occur and plans may change. It is possible (likely, even) that audio traffic is handled just fine by our PEP/PDP architecture: the overall question has always been "how often the PEPs should consult the PDP" and "How long should sessions last?". If sessions last longer than a single request or if the PEP can cache prior PDP responses, then PDP consultation may have an amortized zero impact on traffic latency. If our assumptions are correct and we do need to configure audio channels, the same method may also work for configuring long-running message channels (which solves both of our problems).

For user sign-in, we will use OAuth APIs to enable users to sign in with Google, and we can track their user accounts via the subject ID returned by the OIDC provider (OpenID Connect). User accounts will keep data such as the username, UI options, voice chat volume, and a myriad of other accessibility and customization options. There is the added benefit that session authentication and user password management are offloaded to the OIDC provider, reduces both the complexity and target attractiveness of our back end.

The project will be created with a MERN stack. That is MongoDB, Express, React, and Node. Our frontend will be done with React (a Javascript web framework for modularizing UIs) while our backend will consist of Node (a browserless Javascript runtime environment), Express (a library for easily and modularly handling HTTP/S request), and MongoDB (a NoSQL database that can be easily scaled and replicated). The MERN stack was chosen because it relies on a single language (javascript) throughout, enables modularity, and has a smoother learning curve than some of the alternative options.

II.3.1 Goals

- Keep user data secure: reduce the frequency and scale of data breaches through ZTA, virtualized environments, and widespread encryption
- Allow users to adjust various settings that remain tied to their accounts.
- Provide audio and text messaging services.

II.4. Client and Stakeholder Identification

There aren't any direct stakeholders for this project; Discord is a fantastic piece of software that needs no alternatives. In terms of indirect stakeholders, our development team's grades are on the line, as well as any employment opportunities that may open up on successful completion of this project. Other stakeholders include anyone interested in a student-developed communication service that supports personalized settings, no-download-needed access, and servers that support both text messaging and audio calls. Meat enthusiasts may also present an unexpected (but welcome) steakholder group.

II.5 Team Members & Bios

II.5.1. Member: Cole Godfrey

Cole Godfrey is a Computer Science student primarily interested in machine learning, but also enjoys full stack development. His prior experience includes building a reinforcement learning agent to play Valorant, creating a convolutional neural network to analyze audio waveforms, building a platform which identifies arbitrage opportunities across sportsbooks, and conducting research in safe offline reinforcement learning with a professor and his PhD student. His skills include Python, Pytorch, TypeScript, C/C++, and Node. For this project, his responsibilities are primarily building the various backend microservices, as well as assisting with other miscellaneous responsibilities.

II.5.2. Member: Ethan Goode

Ethan Goode is a Computer Science student interested in microservice architecture, DevOps, and networking. His prior experience includes developing a django-backed satellite tracking website, writing automatic data-collection scripts, and building a home server. His skills include C/C++, project management, and research. For this project his responsibilities include maintaining the server hardware, constructing the CI/CD pipeline, and assisting with microservice development.

II.5.3. Member: Ben Tanabe

Ben Tanabe is a Cybersecurity student interested in security principles, Zero Trust Architecture, and OAuth implementation. His prior experience includes working as a front-end developer for

Lemans Tech on a database for Centers of Independent Living. Ben's skills include Java, JavaScript, HTML, and Python. For this project, he is responsible for the frontend development and assisting with modules as needed.

III. Requirements and Specifications

III.1 Introduction

Beef is a web application for text and audio communication that is highly secure. This is achieved via implementing Zero Trust Architecture and OAuth software. Beef will have administrator accounts for infrastructure deployment and chat moderation. We will use Google's OAuth service to manage our administrator and user credentials. We will allow users to create servers, functioning as large communities where users can text message and audio call one another.

III.2 System Requirements Specification

III.2.1 Use Cases

Login

Pre-condition	- App is open - User is not already logged in
Post-condition	- User is logged in
Basic Path	- User clicks sign in with google - Signs in with the standard google login screen
Alternative Path	
Related Requirements	

Change Username

Pre-condition	- Logged in
Post-condition	- Username is changed
Basic Path	- User clicks on settings -> change username - Types in their new name - Saves new username
Alternative Path	
Related Requirements	

Create Guild

Pre-condition	- Logged in
Post-condition	- A new guild is created with the user as the admin
Basic Path	- Create/Join a guild -> Create a new guild - Enter guild name and any other important details - Click create
Alternative Path	
Related Requirements	- Delete guild, Join guild, Leave guild

Delete Guild

Pre-condition	- Logged in - Admin of guild
Post-condition	- Guild is deleted - Users in the guild are removed from it
Basic Path	- Edit guild button - Delete guild - Confirmation that user does want to delete guild - Delete guild
Alternative Path	
Related Requirements	- Create guild, Join guild, leave guild

Join Guild

Pre-condition	- Logged in
Post-condition	- User is a member of guild
Basic Path	- Create/Join a guild -> Join a guild - Search for the guild - Click join
Alternative Path	- User gets a link to join a guild - User clicks link
Related Requirements	- Delete guild, Create guild, leave guild

Leave Guild

Pre-condition	- Logged in - Member of guild
Post-condition	- User is not apart of the guild
Basic Path	- Guild Options -> Leave guild - Confirmation message “Are you sure you want to leave” - Click leave button
Alternative Path	- User is kicked from guild
Related Requirements	- Delete guild, Edit guild, Join guild

Send Messages in a Guild Channel

Pre-condition	- Logged in - Member of guild
Post-condition	- A new message appears in a guild channel
Basic Path	- Guild -> Channels -> Channel 1 - Type in message - Enter
Alternative Path	
Related Requirements	- Other guild reqs

See Messages in a Guild Channel

Pre-condition	- Logged in - Member of guild
Post-condition	- User can see channel messages
Basic Path	- Guild -> Channels -> Channel 1
Alternative Path	
Related Requirements	- Other guild reqs

Join a voice channel

Pre-condition	- Logged in - Member of guild
Post-condition	- User has joined voice channel - Other users also in the channel can hear them
Basic Path	- Guild -> Channels -> Channel 2 (Voice) - Speak - Hear others response
Alternative Path	
Related Requirements	- Other guild reqs

Control volume of others in voice channel

Pre-condition	- Logged in - Member of guild - In voice channel - Other users in voice channel
Post-condition	- The volume of another user is higher/lower than it was before
Basic Path	- Select a user in the channel - Change sliding bar to modify volume
Alternative Path	- Select a user in the channel - Click mute
Related Requirements	- Send messages, join voice channel

Block a user

Pre-condition	- Logged in - Member of guild
Post-condition	- User has joined voice channel - Other users also in the channel can hear them
Basic Path	- Guild -> Select User -> Block
Alternative Path	
Related Requirements	- Unblock a user

Unblock a user

Pre-condition	- Logged in - Have a blocked user
Post-condition	- User can interact with user 2 again
Basic Path	- Settings -> View blocked users -> Select a specific user -> Unblock - Confirmation
Alternative Path	- Guild -> Select User -> Unblock
Related Requirements	- Block a user

Ban a user entirely from the platform

Pre-condition	- Logged in - Are an administrator of beef, not just a guild admin
Post-condition	- A specific user can no longer access beef.
Basic Path	- Admin panel -> Ban user -> Search for user -> Ban -> Confirm
Alternative Path	
Related Requirements	- Block/unblock/kick user from guild

View any message on the platform

Pre-condition	- User must be logged in - User must be a Platform Administrator
Post-condition	- User can see any message in any channel
Basic Path	- Admin panel -> server search -> select channel
Alternative Path	
Related Requirements	- See messages in guild channel

III.2.2 Functional Requirements

1. Core Application Logic

User account management

Description	The application must contain a system to register and track users. In order to implement the use cases above, we must implement the following fields in MongoDB:
	
	● User SnowflakeID [unique way to identify users within the system]
	● User SubjectID [unique way to identify users with OIDC providers]
	● User DisplayName
	● User Banned? [A setting for app administrators to prevent a user from logging in]
	● Link (or seed) for User Profile Picture (autogenerated)
	● Guild Membership:
	  ○ List of Guilds by Snowflake ID
	● Voice chat settings:
	  ○ Currently active voice channel [may be empty]
	  ○ Self mute [is the user muted by default]
	  ○ Other user volume: [list of the below two]
	  ■ Foreign User Snowflake
	  ■ Foreign User Volume [between 0.0 and 2.0]
	● Blocked users:
	  ○ List of Foreign User Snowflakes
	
	Users can modify their DisplayName, Voice chat settings, guild membership, and blocked users at any point in time. Upon logging in for the first time, users will be assigned a permanent SnowflakeID, SubjectID (by OIDC provider), and a profile picture that will be randomly generated (like the mirrored patterns used by default for github and stack overflow). User authentication is handled by a dedicated OIDC provider as well as a locally hosted authentication service that assigns JSON Web Tokens to users.
	
	When a user is blocked, all messages sent by the blocked user will be hidden from the perspective of the blocking user. When both users join a voice channel together, the blocking user will not be able to hear the blocked user (but will be able to see that they are in the channel). Users can access their own list of blocked users to unblock them at any point.
	
	The user data structure has a link to the currently active voice channel. Since the voice server will be in charge of mixing and relaying audio streams, it is important that the voice server be notified immediately if any user audio settings change. The single voice channel link also makes it impossible for a user to be in two voice channels at the same time.

Priority	Priority Level 0: Core application Logic

Guild Management

Description	The application must contain a system to register, find, join, and delete guilds. In order to support our use cases, we must implement the following fields in MongoDB:
	
	● Guild SnowflakeID [unique way to identify guilds in the system]
	● Guild Name
	● Guild Owner Snowflake ID [which user currently owns this guild]
	● Management requires 2FA? [If disabled, a guild owner can perform administrative actions at any point. Otherwise, the guild owner must first perform an additional verification step]
	● Members:
	  ○ List of members by Snowflake ID
	● Message Channels:
	  ○ List of message channels by snowflake ID
	● Voice channels:
	  ○ List of voice channels by snowflake ID
	● Active invites:
	  ○ List of invite links that can be used to join the guild
	● Banned users:
	  ○ List of users that have been banned [cannot rejoin, even with invite link]
	
	Guilds can be created by any user at any time. Once a guild is created, guild owners can create invites for other users. Guild owners can kick/ban users, transfer ownership, and manage available channels.
	
	When performing an administrative action, the guild backend will check that the user owns the current guild and that they are logged in. The backend will check if the user has already performed privilege escalation if Two-Factor authentication is enabled. If any of the prior steps fail, the User is redirected into a privilege escalation flow.

Priority	<u>Priority Level 0:</u> Core application logic

Message Channel Management:

Description	The application must contain a way to organize, send, and view existing messages. We implement the following fields in MongoDB to do so:
	
	● Channel SnowflakeID [unique way to identify channels]
	● Channel Name
	● Messages:
	  ○ A list of messages, each with the following:
	  ■ Snowflake ID of message
	  ■ Snowflake ID of author
	  ■ Message Content
	  ■ Timestamp

Priority	<u>Priority Level 0:</u> Core application logic

Voice Channel Management

Description	The application must have a way to identify and join active voice channels. The following fields will be provided in MongoDB:
	
	● Channel SnowflakeID
	● Channel Name
	● Active Users:
	  ○ A list of User Snowflake IDs
	● Voice Relay:
	  ○ A unique identifier of the Voice Relay instance that handles audio routing for this channel

Priority	Priority Level 0: Core application logic

Two Factor Authentication

Description	The application can optionally perform a two-step verification process for various administrative actions (the only current administrative actions available for regular users are guild management related). The following items must be stored in MongoDB for successful 2fa: ● Relevant User Snowflake ● 2FA type [only TOTP and None currently] ● Encrypted 2FA metadata: ○ Relevant information to process 2FA request (such as TOTP secret, email, phone number) The 2FA endpoint field allows us to have different 2FA authorization methods by simply specifying a different REST endpoint for each method (e.g. POST /login/challenge/email vs /login/challenge/totp). When a user attempts
Priority	Priority Level 0: Core application logic

2. Additional features

Delete/Edit messages

Description	Being able to fix typos/rephrase messages (or even delete them entirely) can be a useful way to promote communication. In order to facilitate this, we need to add the following to our MongoDB channel schema: ● Edited? [whether or not a message has been edited] ● Old Content: ○ List of prior message values [for moderation purposes] ● Deleted? [For moderation purposes, messages aren't "deleted". They are hidden from non-Platform Administrators if this flag is enabled]
Priority	Priority Level 1: Plan to implement. May be cut if we fall significantly behind schedule

Role-based server management

Description	In order to facilitate community moderation without requiring the guild owner to do everything, role-based moderation can be implemented. It would require the following additions to the guild DB schema:
	
	● Roles:
	  ○ Role name
	  ○ Role permissions:
	  ■ Permission type
	● Permissions:
	  ○ Permission name:value
	  ○ 2FA required?
	● Members:
	  ○ + Roles
	● - 2FA required [instead of a guild-wide boolean, it is now on a permission basis]

Priority	Priority Level 2: Implemented only if time permits

III.2.3 Non-Functional Requirements

Security and consistency- The software should be able to handle user data. It should be consistently accessible, and errors should be quickly addressed. Zero Trust Architecture should be employed to protect user data.

Efficiency- The audio service will function with as much efficiency as the security can allow, through the implementation of one of our audio solutions, which are still in the testing phase. The audio connection should be low-latency and function like a standard audio call.

Accessibility- The system will be easy to access through OAuth, allowing people to easily set up and log in to accounts. The service should function on any browser that can handle JavaScript.

Simplicity- The user interface should be intuitive and easy to use. The options that allow users to create servers, send text messages to the guilds, and start a call/join a voice channel should be clearly defined and simple to access.

III.3 System Evolution

The prototype will be developed with desktop UI in mind and is not designed for mobile. The frontend will be coded with React, which is JavaScript, so the browser will need to be able to support JavaScript.

Rough development timeline:

Back-end	Deadlines
CI/CD pipeline	2/13/2026
Authentication backend	2/27/2026
User service	3/13/2026
Guild service Messaging service	3/27/2026
Zero Trust	4/17/2026

Front-end	Deadlines
Web page presenting service Login page setup	2/27/2026
UI page post login Create and edit user pages Guild sidebar	3/13/2026
Guild UI Messaging UI Theme management/themes	3/27/2026
Polish, ensuring the frontend accommodates all our goals.	4/17/2026

III.4 Standards

Here is a list of standards we plan to at least partially implement for the project. These standards contain requirements to implement other standards, so it would be time-prohibitive to completely implement them in the span of this 3 month project. This list will be updated as the project becomes more mature.

NIST.SP.800-207 - Zero Trust Architecture
RFC 7519 - JSON Web Tokens
RFC 6238 - Time-Based One Time Passwords

IV. Project Solution Approach

IV.1. System Overview

The purpose of this section is to elaborate on the design for our project “Beef”. This section is a guiding document for our development team but isn’t necessarily concrete, but serves as an outline for project outcomes. The goals for our service are as follows:

1. Provide ‘guilds’ the users can use to communicate with each other.
2. Provide a user-friendly interface for creating guilds, managing the volume, muting, and deafening settings of other users.
3. Provide voice and text messaging, user accounts, and all our services under Zero Trust Architecture, to ensure the maximum possible security for the user.

IV.2. Architecture Design

IV.2.1 Overview

The system will need to accommodate the following:

- Guilds where users can interact
- Guild management
- User data management
- Text and voice chat

The system is separated into the following layers to aid in development:

1. Presentation Layer - Displays a GUI for the user and converts user actions into API calls
2. Service Layer - Containerized microservices that receive API calls from the presentation layer. Implements business logic to determine how data should be moved/modified.
3. Data Access Layer - Handles transferring data to/from the database and converting database responses into useable objects.
4. Database Layer - Stores, Indexes, and retrieves data for the Data Access Layer.

For Beef, we are currently considering the following options for each layer:

1. Presentation layer: ReactJS
    - Description: A Javascript library to construct/modularize user interfaces
2. Service Layer: NodeJS with Express
    - Description: A backend javascript runtime environment (NodeJS) coupled with a popular http request-handling framework (Express)
3. Data Access Layer: Mongoose
    - Description: A NodeJS library designed to interface with MongoDB and easily validate data and convert it into objects.
4. Database Layer: MongoDB

- Description: A NoSQL database that stores data in binary JSON format, making it work well with JavaScript objects and the rest of the MERN stack.

IV.2.2 Subsystem Decomposition

Our current design for Beef utilizes a microservice architecture: Every service is defined based on the data structures we use and the actions that need to be performed on that data. Given that the data structures used in this project are not particularly cross-cutting, most of the subsystems practically decomposed themselves.

Service Name	**Service Description**	**Managed data structures**
Auth	In charge of generating session IDs, generating JSON Web Tokens for logged-in users, and performing 2FA to elevate user permissions	User/2FA association, Subject to UserID conversion, and auth Session IDs
Users	Handles all logic associated with User CRUD operations. See section **III.2.2** under **User Account Management.**	User Information
Guilds	Handles all logic associated with Guild CRUD operations, including guild member lists and available channels. See section **III.2.2** under **Guild Management.**	Guild Information
Presenter	Distributes the React web app to clients attempting to reach the site. Also queries the Auth service for Auth Session IDs whenever a user wishes to log in.	React web page, GUI layouts for guilds, users, settings, etc…
Messages	Handles database storage/retrieval of messages and message channel information, as well as relevant CRUD operations. See section **III.2.2** under **Message Channel Management.**	Message Channel Information, Message data
Voice Channels	Handles identifying data for voice channels such as channel name, active voice relay, and currently present users. See section **III.2.2** under **Voice Channel Management**.	Voice Channel Information
Voice Relays	Handles WebRTC communications between members active in a voice channel. This may include audio mixing/muting/distribution.	Unknown. Our team hasn't yet conducted research into what data structures are needed to facilitate WebRTC communication.
Images (?)	We're currently unsure if we want to make	Image data

	this a dedicated service. See note below	

Images Note: For privacy/moderation purposes, users are not currently allowed to upload their own profile images / guild icons. Images will instead be randomly generated for these purposes. The team has not yet decided the scope of such a service (generate only? Generate and store?) or if a dedicated image generation library should be used instead.

IV.3. Data Design

IV.3.1 Database Choice

For this application, MongoDB, an open-source document-oriented NoSQL database is suitable for the following reasons:

1. Schema flexibility:
    - MongoDB stores data in binary JSON format, making it work well with JavaScript objects and the rest of the MERN stack.
    - MongoDB supports schema-less and flexible data structures, allowing us to restructure our data as we continue to develop Beef with little issue.
2. Horizontal scaling:
    - MongoDB supports sharding, a process through which a large data set is broken down into pieces called “shards” and assigned to different machines for storage. By sharding messages by guild or channel, we are able to scale with little to no friction in the event traffic increases.
3. JavaScript Compatibility:
    - We are using the MERN stack, which utilizes JavaScript end-to-end. Since Mongo natively stores data in JSON-like format, we can map our data in MongoDB easily to JavaScript objects through a library like Mongoose.
    - Mongoose allows us to define schemas and validate them; easily translating data in our database to objects we can use in code.

IV.3.2 Database Design

Given this database is being designed in the context of a social chat application similar to Discord, this database is organized into 5 collections: users, guilds, text channels, voice channels, and 2FA. Snowflake IDs are the primary identifier for most documents, which provide unique time-sortable keys for each entry. The following tables describe each collection, its fields, and a description of each field.

Users Collection

Field Name	Type	Description
`userId`	String	Unique snowflake to identify the user
`subjectId`	String	Unique ID assigned by the OIDC Provider to map their external accounts to their beef accounts
`displayName`	String	Display name for the user, can be modified
`banned`	Bool	A flag which prevents login. Default false.
`pictureSeed`	String	A seed used to generate a profile picture
`guildMemberships`	Array of Strings	List of Guild snowflakes the user is a member of
`activeVC`	String | null	Snowflake of the voice channel the user is currently in. Null if not currently in a voice channel.
`muted`	Bool	Flag of whether the user has self-muted (as opposed to being muted by a moderator)
`userVolumes`	Array of Objects	Volume overrides for audio from other users. Each object contains a snowflake of a user (string) and a volume coefficient (from 0.0 to 2.0: mute to 200%, respectively)
`blockedUsers`	Array of Strings	List of blocked User Snowflakes. Messages from these users are hidden and their audio is set to 0 in voice channels.

Guilds Collection

Field Name	Type	Description
`guildId`	String	Unique snowflake to identify the guild
`name`	String	Display name of the guild, modifiable
`ownerId`	String	Snowflake of the user who owns the Guild
`require2FA`	Bool	Whether or not the guild requires two factor authentication to perform administrative actions.
`members`	Array of Strings	List of user snowflakes who are members of the guild.

`messageChannels`	Array of Strings	List of message channel snowflakes that users can see from the guild.
`voiceChannels`	Array of Strings	List of voice channel snowflakes that users can see from the guild
`activeInvites`	Array of Objects	List of invites which contains an invite code (string), validityType (string), validityMetadata (string). The validityType is used to determine how the validityMetadata is interpreted, which in turn decides if the invite can still be used to join the guild.
`bannedUsers`	Array of Strings	List of user snowflakes who are banned from the guild. They cannot rejoin even with a valid invite link.

Message Channels

Field Name	Type	Description
`channelId`	String	Unique snowflake to identify the channel
`name`	String	Display name of the channel
`guildId`	String	Snowflake of the guild the channel is a part of
`messages`	Array of Objects	List of messages. Since there contains many fields within the message, the schema is defined in the next table.

Messages

Field Name	Type	Description
`messageId`	String	Unique snowflake to identify the message
`authorId`	String	Unique user snowflake to identify who sent the message
`content`	String	The actual message
`timestamp`	Date	Timestamp of when the message was created/sent (determined by server not client)

Voice Channels

Field Name	Type	Description

`channelId`	String	Unique snowflake to identify the voice channel
`name`	String	Name of the voice channel
`guildId`	String	Snowflake of the guild the channel is apart of
`activeUsers`	Array of Strings	List of user snowflakes that are currently in the voice channel
`voiceRelayInstance`	String | null	Unique identifier of the currently active Voice Relay associated with the channel (if any).

TwoFactorAuthentication

Field Name	Type	Description
`userId`	String	Unique snowflake to identify the user this 2FA is associated with
`type`	String	The 2FA method that is currently being used (can be “none”)
`encryptedMetadata`	Object	Encrypted blob which contains method-specific data.

IV.4. User Interface Design

The User Interface will be made with React, and the user interface may change as our stretch goals are implemented. Still, for the primary functionality, the user interface will start with our OAuth verification screen, where the user will be prompted to log in with Google.

Once logged in, a user will see their guild sidebar on the left side of their screen, a list of the guilds the user has joined, and the guild at the top of the list, which we refer to as the ‘home guild’, will be automatically opened to the top channel. New users will see the default ‘home guild’ already open to the first channel. If the user has unread messages in a guild, there will be a small indication to the left of the guild icon, as well as one to the left of the specific channel in the guild that contains the unread message. The guilds will have a list of users in the guild to the right of the message box. The guild owner will be indicated by a small icon to the right of their name.

Users will have a settings tab at the bottom left of their screen and a doneness selector, it is a dropdown in the top right of their screen that allows them to choose if they’d like their beef well done, medium, medium rare, or rare.

Inside the server screen, there will be a list of channels to the right of the guild sidebar, the server name will be at the top of the guild sidebar, and the list of channels will be below it. To the right of the guild sidebar will be the text channel, at the bottom of this portion will be a text box to type in, and the list of messages will be timestamped and shown in order of the time they were sent. There will be a guild settings button to the right of the guild name. Voice channels will appear in the guild sidebar, and when a user connects to the voice channel their icon and name will appear under that voice call, and if any other user connects to that voice channel they will appear underneath the user that connection before them, and each user other than the current user will have a settings button in line with their name and icon where a user can manage their muted/deafened status and volume settings.

The user settings page will allow users to change their name and icon, and the guild settings page will allow users to change the notification status of the guild, either ignore or default, as well as a 'leave' option to leave the guild. For admins and the guild owner, the user settings page will also contain the option to create channels, both text and voice, as well as rename and change the icon of the guild. Admins and the owner can click on a user listed on the right of the guild page and get an option to mute their text or voice chat for the whole server, as well as an option to kick and ban a user.

IV.5. Constraints

One of our major constraints is resources; we are using OIDC to protect user data and relying on Google's security, as our team lacks the size, time, and experience to create our own encryption. Our external services are all required to be free, as we do not have funding, and due to our small team, our internal services should be manageable by a single person or at most a small team. Our team does not have a lot of experience using a MERN stack and even less with Zero Trust Architecture systems, so we need to dedicate some time to learning and experimenting to deepen our understanding of these concepts. Another constraint would be voice chat; we need to test a solution to make voice chat work with a Zero Trust Architecture design, without having the voice chat be unusably laggy. Additionally, as upscaling occurs and more people are brought onto the platform, there will be difficulty in scaling up the software to manage the increased userbase. Things like loading messages from years ago and handling rare edge cases, such as two people sending a message into the same channel at the same millisecond, will become a concern as the program increases in userbase. Users who are unwilling to go through Google will be unable to access our service; this is a constraint of using OIDC to manage logging in. We are creating in a very saturated field, so avoiding stepping on any patents, such as the Seamless Switching patent and the AI Personal Agent patent are an additional constraint. Additionally, designing a solution to log out is a constraint, as we take users in with an access token, which is given to us by the Auth and has an expiration date, but to make the token invalidate early, we need to constantly check the validation of the token in some fashion, which complicates our OIDC so a solution may be necessary to handle user logout.

Styles can be applied using the top menu: Format > Paragraph Styles > (Heading 1, Heading 2, …)

TITLE

Subtitle

I. Heading 1

I.1. Heading 2

Normal Text